

**Московский государственный технический
университет им. Н.Э. Баумана**

**Факультет «Радиотехнический»
Кафедра «Системы обработки информации и управления»**

Курс «Парадигмы и конструкции языков программирования»

**Отчет по лабораторной работе №2-3
«Функциональные возможности языка Python»**

Выполнил:
студент группы РТ5-31Б:
Кузнецов С. А.

Подпись и дата:

Проверил:
преподаватель кафедры ИУ5
Гапанюк Ю. Е.

Подпись и дата:

Москва, 2024 г.

Постановка задачи

Задание лабораторной работы состоит из решения нескольких задач.

Файлы, содержащие решения отдельных задач, должны располагаться в пакете `lab_python_fr`. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл `field.py`)

Необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря. Пример:

```
goods = [  
    {'title': 'Ковер', 'price': 2000, 'color': 'green'},  
    {'title': 'Диван для отдыха', 'color': 'black'}  
]
```

`field(goods, 'title')` должен выдавать 'Ковер', 'Диван для отдыха'

`field(goods, 'title', 'price')` должен выдавать `{'title': 'Ковер', 'price': 2000}`, `{'title': 'Диван для отдыха'}`

В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.

Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.

Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

Задача 2 (файл `gen_random.py`)

Необходимо реализовать генератор `gen_random`(количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:

`gen_random(5, 1, 3)` должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Задача 3 (файл `unique.py`)

Необходимо реализовать итератор `Unique(данные)`, который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.

Конструктор итератора также принимает на вход именованный `bool`-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`.

При реализации необходимо использовать конструкцию `**kwargs`.

Итератор должен поддерживать работу как со списками, так и с генераторами.

Итератор не должен модифицировать возвращаемые значения.

Пример:

```
data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
```

`Unique(data)` будет последовательно возвращать только 1 и 2.

```
data = gen_random(10, 1, 3)
```

`Unique(data)` будет последовательно возвращать только 1, 2 и 3.

```
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
```

`Unique(data)` будет последовательно возвращать только a, A, b, B.

`Unique(data, ignore_case=True)` будет последовательно возвращать только a, b.

Задача 4 (файл `sort.py`)

Дан массив 1, содержащий положительные и отрицательные числа. Необходимо одной строкой кода вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции `sorted`. Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
```

Вывод: `[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]`

Необходимо решить задачу двумя способами:

- С использованием `lambda`-функции.
- Без использования `lambda`-функции.

Задача 5 (файл `print_result.py`)

Необходимо реализовать декоратор `print_result`, который выводит на экран результат выполнения функции.

Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.

Если функция вернула список (`list`), то значения элементов списка должны выводиться в столбик.

Если функция вернула словарь (`dict`), то ключи и значения должны выводиться в столбик через знак равенства.

Задача 6 (файл `cm_timer.py`)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():
```

```
    sleep(5.5)
```

После завершения блока кода в консоль должно вывестись `time: 5.5` (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Задача 7 (файл `process_data.py`)

В предыдущих задачах были написаны все требуемые инструменты для работы с данными. Применим их на реальном примере.

В файле `data_light.json` содержится фрагмент списка вакансий.

Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.

Необходимо реализовать 4 функции - `f1`, `f2`, `f3`, `f4`. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора `@print_result` печатается результат, а контекстный менеджер `cm_timer_1` выводит время работы цепочки функций.

Предполагается, что функции `f1`, `f2`, `f3` будут реализованы в одну строку. В реализации функции `f4` может быть до 3 строк.

Функция f1 должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.

Функция f2 должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию filter.

Функция f3 должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию map.

Функция f4 должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист C# с опытом Python, зарплата 137287 руб. Используйте zip для обработки пары специальность — зарплата.

Текст программы

Файл «field.py»:

```
# Пример:
# goods = [
#     {'title': 'Ковер', 'price': 2000, 'color': 'green'},
#     {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
# ]
# field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'
# field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price': 2000},
# {'title': 'Диван для отдыха', 'price': 5300}

def field(items: list[dict], *args):
    argcnt = len(args)

    # items - список словарей, position - словарь из списка, fld - поле из переданных аргументов
```

```

if argcnt == 1:
    for position in items:
        for fld in args:
            yield position.get(fld)
else:
    for position in items:
        searchres = {fld: position[fld] for fld in args}
        if searchres:
            yield searchres

if __name__ == "__main__":
    goods = [
        {"title": "Ковер", "price": 2000, "color": "green"},
        {"title": "Диван для отдыха", "price": 5300, "color": "black"},
    ]

    for f in field(goods, "title"):
        print(f)

    for f in field(goods, "title", "price"):
        print(f)

```

Файл «gen_random.py»:

```

import random

# Пример:
# gen_random(5, 1, 3) должен выдать 5 случайных чисел
# в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

```

```
def gen_random(num_count, begin, end):
    random.seed()
    return [random.randrange(begin, end + 1) for i in range(num_count)]

if __name__ == "__main__":
    print(gen_random(5, 1, 3))
```

Файл «unique.py»:

```
# Итератор для удаления дубликатов

class Unique(object):
    # Bool-параметр ignore_case определяет, будут ли считаться одинаковыми
    строки в разном
    # регистре.
    # Например: ignore_case = True, Абв и АБВ - разные строки
    # ignore_case = False, Абв и АБВ - одинаковые строки, одна из кото-
    рых удалится
    # По-умолчанию ignore_case = False

    def __init__(self, items, **kwargs):
        self.data = items
        self.used_elements = set()
        self.index = 0

        if kwargs.get("ignore_case") != None:
            self.ignore_case = kwargs.get("ignore_case")
        else:
            self.ignore_case = False

    def __next__(self):
        while True:
```

```

    if self.index >= len(self.data):
        raise StopIteration
    else:
        current = self.data[self.index] # текущий элемент из коллекции
        self.index = self.index + 1 # следующий за ним номер
        if (current not in self.used_elements and self.ignore_case == True) or (
            str(current).casefold() not in self.used_elements
            and self.ignore_case == False
        ):
            if self.ignore_case == False:
                self.used_elements.add(str(current).casefold())
                # при ignore_case = False множество уникальных элементов со-
                # держит строки только в нижнем регистре
            else:
                self.used_elements.add(current)
        return current

def __iter__(self):
    return self

if __name__ == "__main__":
    data = ["a", "A", "b"]

    for item in Unique(data, ignore_case=True):
        print(item)

```

Файл «sort.py»:

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]
```



```

if __name__ == "__main__":
    result = sorted(data, key=abs, reverse=True)
    print(result)

    result_with_lambda = sorted(data, key=lambda x: x if x >= 0 else -x, reverse=True)
    print(result_with_lambda)

```

Файл «print_result.py»:

```

def print_result(func):

    def printer(*args, **kwargs):
        func_res = func(*args, **kwargs)
        print(func.__name__)
        if type(func_res) is dict:
            for key, value in func_res.items():
                print(f"{key} = {value}")
        else:
            if type(func_res) is list:
                for item in func_res:
                    print(item)
            else:
                print(func_res)
        return func_res

    return printer

@print_result
def test_1():

```

```
    return 1

@print_result
def test_2():
    return "iu5"

@print_result
def test_3():
    return {"a": 1, "b": 2}

@print_result
def test_4():
    return [1, 2]

if __name__ == "__main__":
    print("!!!!!!!")
    test_1()
    test_2()
    test_3()
    test_4()
```

Файл «cm_timer.py»:

```
import time
from contextlib import contextmanager

# Контекстный менеджер, измеряющий время работы блока кода
```

```
class cm_timer_1:

    def __init__(self):
        self.exec_start = 0
        self.exec_end = 0

    def __enter__(self):
        self.exec_start = time.time()

    def __exit__(self, type, value, traceback):
        self.exec_end = time.time()
        print(f"time: {self.exec_end - self.exec_start}")

@contextmanager
def cm_timer_2():
    exec_start = time.time()
    try:
        yield exec_start
    finally:
        print(f"time: {time.time() - exec_start}")

if __name__ == "__main__":
    with cm_timer_1():
        time.sleep(5.5)
```

```
with cm_timer_2():  
    time.sleep(5.5)
```

Файл «process_data.py»:

```
import json  
import sys  
from field import field  
from gen_random import gen_random  
from unique import Unique  
from print_result import print_result  
from cm_timer import cm_timer_1  
  
path = "/home/gato/Документы/funcpy/data_light.json"  
  
# Необходимо в переменную path сохранить путь к файлу, который был пере-  
дан при запуске сценария  
  
with open(path) as f:  
    data = json.load(f)  
  
# Предполагается, что функции f1, f2, f3 будут реализованы в одну строку  
# В реализации функции f4 может быть до 3 строк  
  
@print_result  
def f1(arg):  
    result = Unique(sorted(field(arg, "job-name"), reverse=True), ig-  
nore_case=True) # type: ignore  
    return result
```

```

@print_result
def f2(arg):
    return filter(lambda x: True if "программист" in x else False, arg)

@print_result
def f3(arg):
    return map(lambda x: x + " с опытом Python", arg)

@print_result
def f4(arg):
    list_from_arg = list(arg)
    return dict(zip(list_from_arg, gen_random(len(list_from_arg), 100000,
200000)))

if __name__ == "__main__":
    with cm_timer_1():
        f4(f3(f2(f1(data))))

```

Экранные формы с примерами выполнения программы

Файл «field.py»:

```

Ковер
Диван для отдыха
{'title': 'Ковер', 'price': 2000}
{'title': 'Диван для отдыха', 'price': 5300}
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>

```

Файл «gen_random.py»:

```
[2, 1, 2, 2, 3]  
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «unique.py»:

```
a  
A  
b  
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «sort.py»:

```
[123, 100, -100, -30, 4, -4, 1, -1, 0]  
[123, 100, -100, -30, 4, -4, 1, -1, 0]  
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «print_result.py»:

```
!!!!!!!  
test_1  
1  
test_2  
iu5  
test_3  
a = 1  
b = 2  
test_4  
1  
2  
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «cm_timer.py»:

```
time: 5.500240087509155  
time: 5.500479221343994  
PS C:\Users\User\Documents\BiquadraticEquations\PCPL>
```

Файл «process_data.py»:

```
f1
<unique.Unique object at 0x000001BDA8F58D70>
f2
<filter object at 0x000001BDA6BF0AC0>
f3
<map object at 0x000001BDA6BF0AF0>
f4
программист 1С с опытом Python = 107901
программист с опытом Python = 151448
педагог программист с опытом Python = 111491
инженер-программист с опытом Python = 175170
инженер - программист с опытом Python = 136678
веб-программист с опытом Python = 189994
Старший программист с опытом Python = 158592
Системный программист (С, Linux) с опытом Python = 115564
Помощник веб-программиста с опытом Python = 143666
Инженер-электронщик (программист АСУ ТП) с опытом Python = 194821
Инженер-программист САПОУ (java) с опытом Python = 105093
Инженер-программист ПЛИС с опытом Python = 185316
Инженер-программист ККТ с опытом Python = 103439
Инженер-программист 1 категории с опытом Python = 113477
Инженер-программист (Орехово-Зуевский филиал) с опытом Python = 129087
Инженер-программист (Клинский филиал) с опытом Python = 135418
Инженер-программист с опытом Python = 134203
Инженер - программист АСУ ТП с опытом Python = 128266
Ведущий программист с опытом Python = 156572
Ведущий инженер-программист с опытом Python = 165404
Веб-программист с опытом Python = 124528
Веб - программист (PHP, JS) / Web разработчик с опытом Python = 196574
Web-программист с опытом Python = 166916
1С программист с опытом Python = 118834
time: 0.05558466911315918
PS C:\Users\User\Documents\BiquadraticEquations\PCPL> 
```